COLLEGE OF ENGINEERING, GUINDY, ANNA UNIVERSITY

INTERNSHIP REPORT

DURATION – 1st June to 31st July

SUBMITTED BY:

CHARAN MS
2019110004
Third year
COLLEGE OF ENGINEERING, GUINDY, ANNA UNIVERSITY

SUBMITTED TO:

Dr. –Ing PRINCE GIDEON KUBENDRAN AMOS
ASSISTANT PROFESSOR
DEPARTMENT OF METALLURGICAL AND MATERIALS ENGINEERING
NATIONAL INSTITUTE OF TECHNOLOGY, TIRUCHIRAPPALI

ABSTRACT

This report contains the work done during the internship period under Assistant Professor, Prince Gideon Kubendran Amos, National Instiute of Technology, Tiruchirappalli (NITT). The report shows an overview of the tasks done during the internship in detail. The purpose of this report is to explain what I did and learned during the internship period from 1st June to 31st July 2021. The report explains the numerical method of solving mathematical models using Finite Difference Method. There is also insight into the various boundary conditions that can be applied while solving them. We also worked on the basics of Phase Field Modelling by using Finite Difference Method for Dissolution of Pearlite with Fick's Law of Diffusion as the mathematical model.

.

TABLE OF CONTENTS

S.NO.	PARTICLERS
1.	PARTIAL DIFFERENTIAL EQUATIONS
2.	CLASSIFICATION OF PDE
3.	INITIAL AND BOUNDARY CONDITIONS
4.	FINITE DIFFERENCE METHOD
5.	DIRICHLET BOUNDARY CONDITION
5.1	1D HEAT EQUATION - FDM AND DIRICHLET BC
5.2	1D HEAT EQUATION – FDM AND DIRICHLET BC WITH ZERO BOUNDARY CONDITION
5.3	1D HEAT EQUATION – FDM AND DIRICHLET BC WITH NON-ZERO BOUNDARY CONDITION
6.	NEUMANN BOUNDARY CONDITION
6.1	1D HEAT EQUATION – FDM AND NEUMANN BC-ZERO FLUX AT BOTH BOUNDARIES - 1
6.2	1D HEAT EQUATION – FDM AND NEUMANN BC-NON-ZERO FLUX AT BOTH BOUNDARIES - 1
6.3	1D HEAT EQUATION – FDM AND NEUMANN BC-ZERO FLUX AT BOTH BOUNDARIES - 2
6.4	1D HEAT EQUATION – FDM AND NEUMANN BC-NON-ZERO FLUX AT BOTH BOUNDARIES - 2
7.	1D HEAT EQUATION – FDM AND MIXED BC (DIRICHLET AND NEUMANN)
8.	THE 2D HEAT EQUATION
8.1	2D HEAT EQUATION – FDM AND DIRICHLET BC
8.2	2D HEAT EQUATION – FDM AND DIRICHLET BC (CODE 2)
8.3	2D HEAT EQUATION – FDM AND MIXED BC (DIRICHLET AND NEUMANN)
9	INTRODUCTION TO PHASE FIELD MODELLING
10	DISSOLUTION OF PEARLITE
10.1	NUMERICAL MODELLING OF PEARLITE DISSOLUTION - DIRICHLET BOUNDARY CONDITION
10.2	NUMERICAL MODELLING OF PEARLITE DISSOLUTION – NEUMANN BOUNDARY CONDITION

1. PARTIAL DIFFERENTIAL EQUATIONS

- In mathematics, a partial differential equation (PDE) is an equation that imposes relations between the various partial derivatives of a multivariable function.

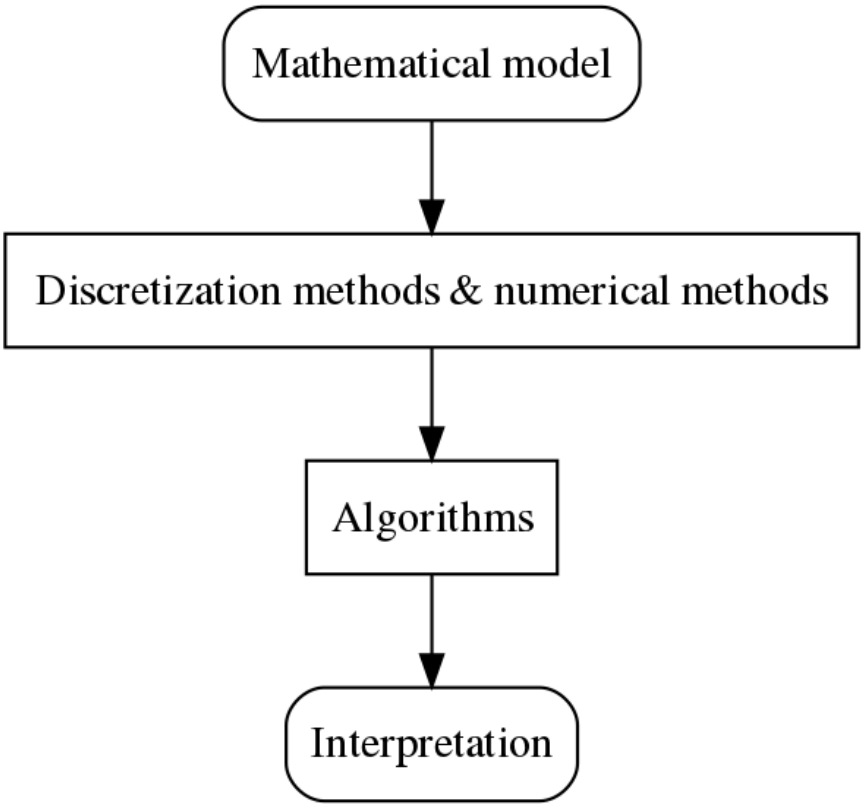
- Partial differential equations are ubiquitous in mathematically oriented scientific fields, such as physics and engineering. For instance, they are foundational in the modern scientific understanding of sound, heat, diffusion, electrostatics, electrodynamics, fluid dynamics, elasticity, general relativity, and quantum mechanics.

- The heart of analysis that utilize fundamental physics-based principles are differential equations. In the most general case, when the behavior of the system or device is sought as a function of both time and space, the governing equations are PDEs.

- The solution of a PDE – either by analytical or numerical means – is generally quite complex, or requires a deep understanding of the key attributes of the PDE. These attributes dictate the basic method of solution, how and where boundary and initial conditions must be applied, and what the general nature of the solution is expected to be.

- We can use numerical methods to solve some partial differential equations (PDEs) in general. By solving them numerically and visualizing the solutions as a heat map we get a deeper understanding of the significance of the equations.

Approach to solve a Partial Differential Equation



- 1D Heat Equation, 2D Heat Equation and Fick's Law of Diffusion (For Pearlite Dissolution) have been solved during the course of this internship.

2. CLASSIFICATION OF PDE

A partial differential equation of the form

$$Au_{xx} + Bu_{xy} + Cu_{yy} = f(x, y, u, u_x, u_y)$$

where *x* and *y* are so-called independent variables, while *u* is the dependent variable. The coefficients *A, B*, and *C* are either real constants or real functions of the independent or dependent variables. If any of the three coefficients is a function of the dependent variable *u*, the PDE becomes nonlinear. Otherwise, it is linear. The above equation is a PDE in two independent variables. In general, of course, PDEs can be in more than two independent variables.

Depending on the values of the coefficients *A, B*, and *C*, PDEs are classified as follows:

If $B^2 - 4AC < 0$, then the PDE is **elliptic**
ie $U_{xx} + U_{yy} = f(x,y)$ (Poisson's equation and Laplace's equation when $f(x,y) = 0$)

If $B^2 - 4AC = 0$, then the PDE is **parabolic**
ie $U_t = kU_{xx}$ (1D diffusion equation)

If $B^2 - 4AC > 0$, then the PDE is **hyperbolic**.
ie $U_{tt} = C^2 U_{xx}$ (Wave equation)

3. <u>INITIAL AND BOUNDARY CONDITIONS</u>

- PDEs require proper initial conditions (ICs) and boundary conditions (BCs) to solve.

- A second order elliptic PDE ($U_{xx} + U_{yy} = f(x,y)$) requires a boundary condition on *u* at each point in the boundary. Thus these are called Boundary-Value (BV) problems. The Boundary condition maybe a value of *u* on the boundary or the value of its derivative.

- Linear Parabolic Equations ($U_t = kU_{xx}$) require ICs at the initial start time (t=0) and one BC at each end point of the spatial domain.

- Hyperbolic Equations ($U_{tt} = C^2 U_{xx}$) require ICs and as many BCs as there are inward pointing characteristics which depend on the sign of wave speed 'C'. Thus

  If C>0

  IC: U (0, x) = f(x); **BC**: U (t, 0) = g (t)

  If C<0

  IC: U (0, x) = f(x); no **BC**

4. FINITE DIFFERENCE METHOD

- The finite difference method is based on the calculus of finite differences.

- It is a method in which the governing PDE is satisfied at a set of prescribed interconnected points within the computational domain, referred to as *nodes*.

- The boundary conditions, in turn, are satisfied at a set of prescribed nodes located on the boundaries of the computational domain. The framework of interconnected nodes is referred to as a *grid* or *mesh*.

- Each derivative in the PDE is approximated using a difference approximation that is typically derived using Taylor series expansions.

TAYLOR'S THEOREM:

Let U(x) have n continuous derivatives over the interval (a,b). Then for a $< x_0, x_{0+h} <$ b

$$U(x_o+h)=U(x_o)+hU_x(x_o)+h^2\frac{U_{xx}(x_o)}{2!}+\ldots+h^{n-1}\frac{U_{(n-1)}(x_o)}{(n-1)!}+O(h^n)$$

where

$$U_x=\frac{dU}{dx}, U_{xx}=\frac{d^2U}{dx^2},\ldots,U_{(n-1)}=\frac{d^{n-1}U}{dx^{n-1}}$$

- $U_x(x_0)$ is the derivative of U with respect to x evaluated at $x = x_0$
- $O(h^n)$ is an unknown error term.

The interpretation of Taylor's Theorem is that if we know the value of U and the values of its derivatives at point x_0 then we can find its value at the point $x_0 + h$. If we truncate the right hand side of the above equation (Discard the term $O(h^n)$) we get an approximation to $U(x_0 + h)$. The error in this approximation is $O(h^n)$.

TAYLOR'S THEOREM APPLIED TO FDM:

- In FDM we know the U value at the grid points and we want to replace partial derivatives in the PDE we are solving by approximations at these grid points.

- In FDM both x_0 and $x_0 + h$ are grid points and $U(x_0)$ and $U(x_0 + h)$ are known. Thus we can rearrange the above equation to get Finite Difference approximations to derivatives which have $O(h^n)$ errors.

SIMPLE FD APPROXIMATION TO A DERIVATIVE:

$$U(x_o+h)=U(x_o)+hU_x(x_o)+h^2\frac{U_{xx}(x_o)}{2!}+\ldots+h^{n-1}\frac{U_{(n-1)}(x_o)}{(n-1)!}+O(h^n)$$

Truncating the above equation after first derivative gives

$$U(x_o+h)=U(x_o)+hU_x(x_o)+O(h^2) \quad \text{---------- } 1$$

Rearranging equation 1 gives

$$U_x(x_o)=\frac{U(x_o+h)-U(x_o)}{h}+\frac{O(h^2)}{h}$$

$$=\frac{U(x_o+h)-U(x_o)}{h}+O(h) \quad \text{---------- } 2 \qquad f(h)/h = O(h^{n-1})$$

Neglecting the O (h) term gives

$$U_x(x_o) \approx \frac{U(x_o+h)-U(x_o)}{h} \quad \text{----------}3$$

Equation 3 is the first order FD approximation to $U_x(x_0)$ since the approximation error O (h) depends on the first power of h. This approximation is called forward FD approximation since we start at x_0 and step forward to the point $x_0 + h$. h is called the step size.

FINITE DIFFERENCE APPROXIMATIONS:

Let's assume that U is a function of only 2 variables, t and x. We approximate the partial derivatives of U w.r.t x. As t is held constant U is a function of the single variable x and so we can use Taylor's formula where

the ordinary derivative terms are now partial derivatives and the arguments are (t,x) instead of x. We replace the step size h by Δx to indicate a change in x so that the equation becomes

$$U(t, x_o + \Delta x) = U(t, x_o) + \Delta x U_x(t, x_o) + \frac{\Delta x^2}{2!} U_{xx}(t, x_o) + \ldots + \frac{\Delta x^{n-1}}{(n-1)!} U_{(n-1)}(t, x_o) + O(\Delta x^n)$$

Truncating the above equation to $O(\Delta x^2)$ we get

$$U(t, x_o + \Delta x) = U(t, x_o) + \Delta x U_x(t, x_o) + O(\Delta x^2)$$

We now derive some FD approximations to partial derivatives. Rearranging the above equation we get

$$U_x(t, x_o) = \frac{U(t, x_o + \Delta x) - U(t, x_o)}{\Delta x} - \frac{O(\Delta x^2)}{\Delta x}$$

$$\therefore\ U_x(t, x_o) = \frac{U(t, x_o + \Delta x) - U(t, x_o)}{\Delta x} - O(\Delta x)$$

For solving PDEs we are restricted to a grid of discrete x values $x_1, x_2 \ldots x_n$ and discrete t levels $t_0, t_1, t_2 \ldots$..We will assume a constant grid spacing Δx in x so that $x_{i+1} = x_i + \Delta x$. Evaluating the above equation for a point (t_n, x_i) on the grid gives,

$$U_x(t_n, x_i) = \frac{U(t_n, x_{i+1}) - U(t_n, x_i)}{\Delta x} - O(\Delta x)$$

$$U_i^n = U(t_n, x_i)$$

Dropping the $O(\Delta x)$ error term we get,

$$U_x(t_n, x_i) \approx \frac{U_{i+1}^n - U_i^n}{\Delta x}$$

This is the first order forward difference approx. to U_x (t_n, x_i).

Replacing Δx by $-\Delta x$ gives

$$U_x(t_n, x_i) \approx \frac{U_i^n - U_{i-1}^n}{\Delta x}$$

This is the first order backward difference approx. to U_x (t_n, x_i).

We can increase the order by taking in more terms in the Taylor's series so as to make the approximation more accurate.

$$U_x(t_n, x_i) \approx \frac{U_{i+1}^n - U_{i-1}^n}{2\Delta x}$$

The above expression is the second order central difference FD approximation to U_x (t_n, x_i).

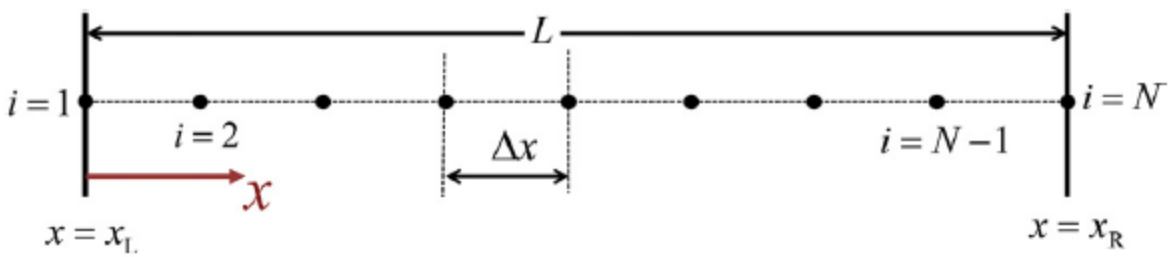
FD approximations to partial derivatives with respect to't' and 'x' are represented in a table format:

partial derivative	finite difference approximation	type	order
$\frac{\partial U}{\partial x} = U_x$	$\frac{U_{i+1}^n - U_i^n}{\Delta x}$	forward	first in x
$\frac{\partial U}{\partial x} = U_x$	$\frac{U_i^n - U_{i-1}^n}{\Delta x}$	backward	first in x
$\frac{\partial U}{\partial x} = U_x$	$\frac{U_{i+1}^n - U_{i-1}^n}{2\Delta x}$	central	second in x
$\frac{\partial^2 U}{\partial x^2} = U_{xx}$	$\frac{U_{i+1}^n - 2U_i^n + U_{i-1}^n}{\Delta x^2}$	symmetric	second in x
$\frac{\partial U}{\partial t} = U_t$	$\frac{U_i^{n+1} - U_i^n}{\Delta t}$	forward	first in t
$\frac{\partial U}{\partial t} = U_t$	$\frac{U_i^n - U_i^{n-1}}{\Delta t}$	backward	first in t
$\frac{\partial U}{\partial t} = U_t$	$\frac{U_i^{n+1} - U_i^{n-1}}{2\Delta t}$	central	second in t
$\frac{\partial^2 U}{\partial t^2} = U_{tt}$	$\frac{U_i^{n+1} - 2U_i^n + U_i^{n-1}}{\Delta t^2}$	symmetric	second in t

5. DIRICHLET BOUNDARY CONDITION

In Dirichlet BC the value of the dependent variable, u, is prescribed on the boundary.

In the finite difference method, since nodes are located on the boundary, the Dirichlet boundary condition can be easily applied.



$u_1 = u_L$ (The value of u at the first node is same as the value of u at the left boundary)

$u_N = u_R$ (The value of u at the final node N is same as the value of u at the right boundary)

THE 1D HEAT EQUATION

$$\frac{\partial u}{\partial t} = c^2 \frac{\partial^2 u}{\partial x^2}$$

- C^2 is called the thermal diffusivity of the rod.
- u is the temperature, t is time and x is space.

5.1 1D HEAT EQUATION - FDM AND DIRICHLET BC

- We first start with diffusion equation with initial condition $u(x, 0) = 4x - 4x^2$ and $u(0, t) = u(L, t) = 0$
- First discretize the domain [0, L] X [0, T]
- Then discretize the derivatives

Generating algorithm:

1. Compute initial condition for all i
2. For all n:

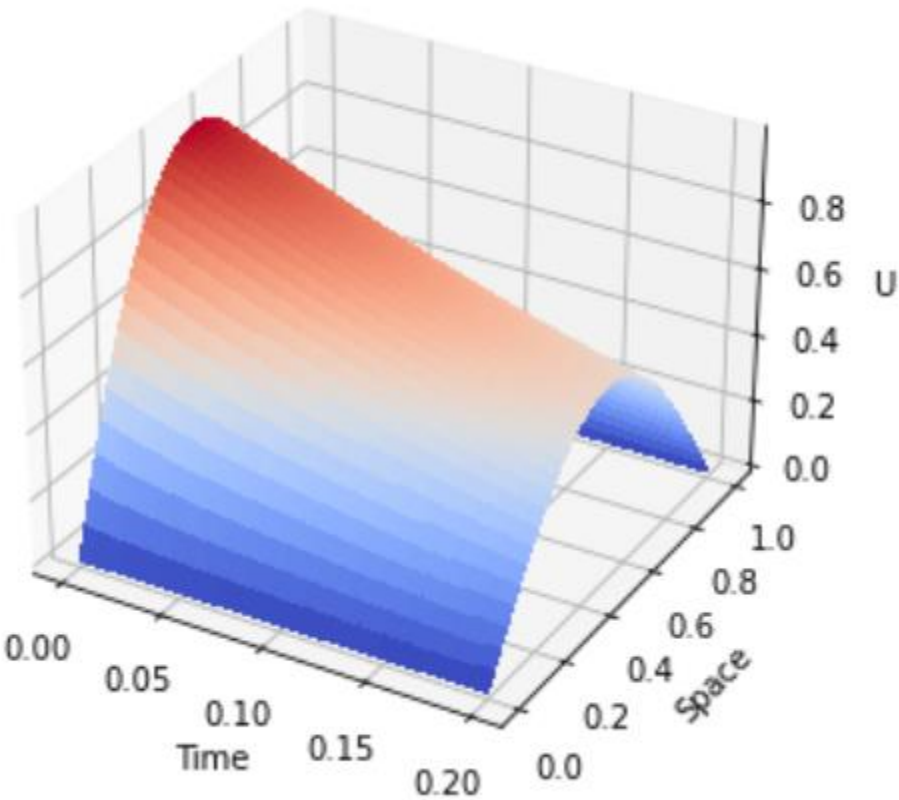
2i. Compute $u_i^{\{n+1\}}$ for internal space points

2ii. Set boundary values for i = 0 and i = N_x

```
M = 40 # number of grid points for space interval
N = 200 #number of grid points for time interval

x0 = 0
xL = 1 # unit grid differences

dx = (xL - x0) / (M - 1) # space step

t0 = 0
tF = 0.2

dt = (tF - t0) / (N - 1)

D = 0.3 # thermal diffusivity

a = D * dt / dx**2

# Create grid
tspan = np.linspace(t0, tF, N)
xspan = np.linspace(x0, xL, M)


# Initial matrix solution
U = np.zeros((M, N))
```

```python
# Initial condition
U[:, 0] = 4*xspan - 4*xspan**2
# Boundary conditions
U[0, :] = 0
U[-1, 0] = 0

# Discretised derivative formula
for k in range(0, N-1):
    for i in range(1, M-1):
        U[i, k+1] = a * U[i-1, k] + (1 - 2 * a) * U[i, k] + a * U[i + 1, k]
```

Output (3D Representation):



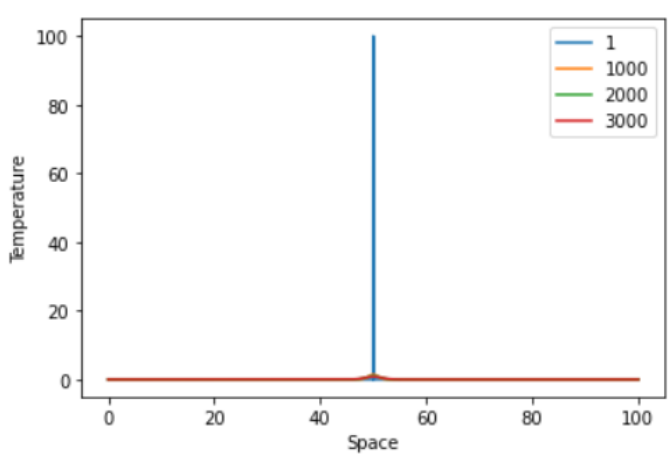
5.2 1D HEAT EQUATION – FDM AND DIRICHLET BC WITH ZERO BOUNDARY CONDITION

Constant initial condition and zero BC at both ends

```
t_vec=np.linspace(t0,tL,N)   #creating grid
x_vec=np.linspace(x0,xL,M)   #creating grid
T=np.zeros((M,N))             #inital matrix values

bc1=float(input("Enter bounday value at x0 for any t:"))
bc2=float(input("Enter bounday value at xL for ant t:"))
T[0,:]=bc1
T[M-1,:]=bc2

T[1500] = 100   #initial condition

for n in range(0,N-1):
    for i in range(1,M-1):
        T[i,n+1] = (alpha*( T[i-1,n] + T[i+1,n] )) + ((1-2*alpha)*T[i,n])
```

Inputs and Output (2D representation):

```
Enter No. of grid points for space interval:3000
Enter No. of grid points for time interval:3000
Enter starting space grid point:0
Enter final space grid point:100
Enter starting time grid point:0
Enter final time grid point:10
Enter Thermal Diffusivity value:0.005
Enter bounday value at x0 for any t:0
Enter bounday value at xL for ant t:0
```



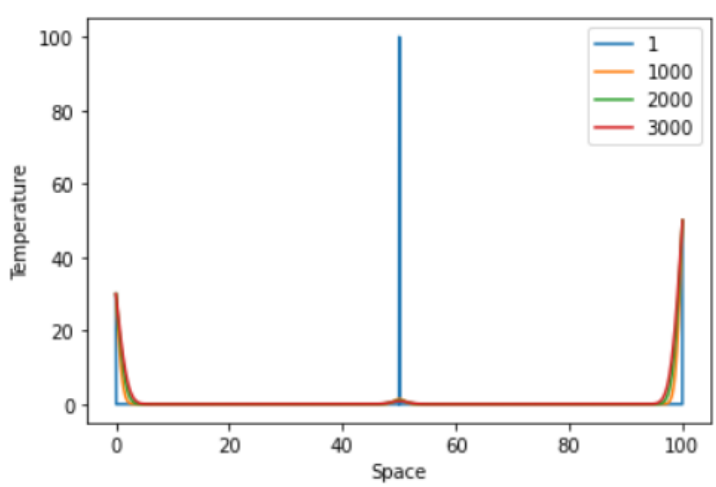
5.3 1D HEAT EQUATION – FDM AND DIRICHLET BC WITH NON-ZERO BOUNDARY CONDITION

Constant initial condition and non-zero BC at both ends

```
t_vec=np.linspace(t0,tL,N)   #creating grid
x_vec=np.linspace(x0,xL,M)   #creating grid
T=np.zeros((M,N))             #inital matrix values

bc1=float(input("Enter bounday value at x0 for any t:"))
bc2=float(input("Enter bounday value at xL for ant t:"))
T[0,:]=bc1
T[M-1,:]=bc2

T[1500] = 100   #initial condition

for n in range(0,N-1):
    for i in range(1,M-1):
        T[i,n+1] = (alpha*( T[i-1,n] + T[i+1,n] )) + ((1-2*alpha)*T[i,n])
```

Inputs and Output (2D representation):

```
Enter No. of grid points for space interval:3000
Enter No. of grid points for time interval:3000
Enter starting space grid point:0
Enter final space grid point:100
Enter starting time grid point:0
Enter final time grid point:10
Enter Thermal Diffusivity value:0.005
Enter bounday value at x0 for any t:30
Enter bounday value at xL for ant t:50
```

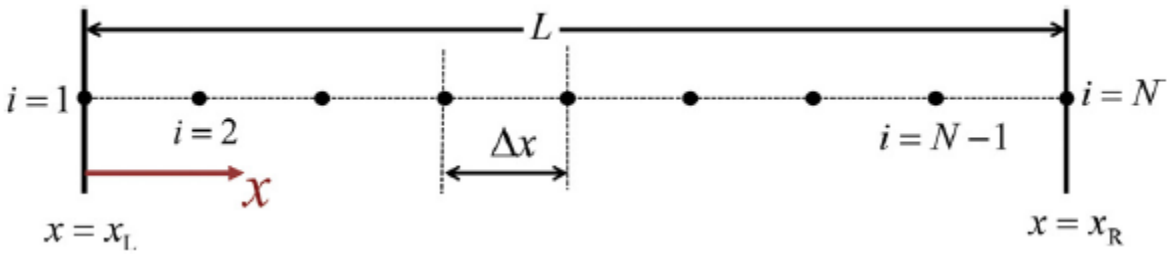


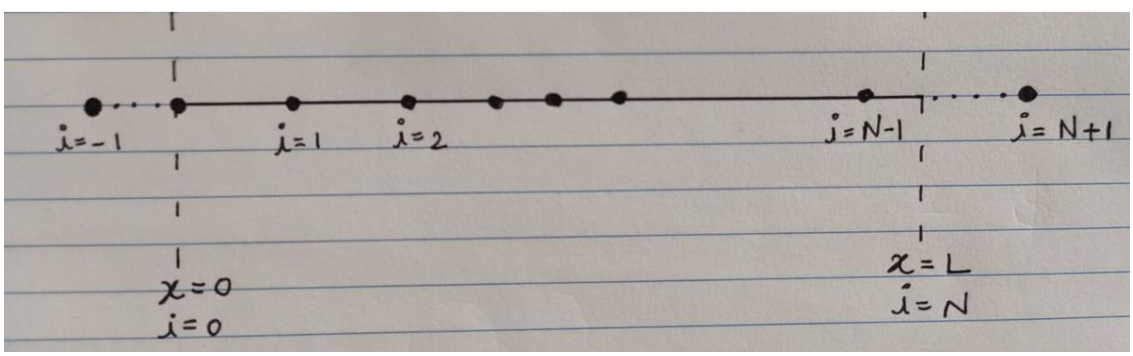
6. NEUMANN BOUNDARY CONDITION

- Boundary conditions that specify the value of $\partial u / \partial n$ (or shorter u_n) are known as Neumann conditions

- The first derivative is a measure of the flux at the boundary.

- Neumann boundary condition is often referred to as a flux boundary condition.





- At i = 0 and i = N Neumann B.C. is applied.

- We introduce 2 ghost boundary points (points outside the mesh) i.e. i = -1 and i = N+1. We use them to set up the equation at the boundary.

- We use the central difference approximation at the boundaries and get an expression for the ghost points.

- We then substitute this expression in the explicit Euler time step equation.

Example: Heat Equation with Neumann B.C.

$$u_x(t,0) = 0 \; ; \quad u_x(t,1) = \alpha \quad , \quad t \geq 0$$

Explicit Euler time step

$$u_i^{n+1} = \gamma u_{i-1}^n + (1-2\gamma) u_i^n + \gamma u_{i+1}^n \quad \text{— (1)} \qquad i = 1, 2, \ldots M-1$$

Introduce 2 ghost boundary points

$$x_{-1} = -h \; ; \quad x_{M+1} = x_M + h = 6 + h$$

$$u_{-1}^n \approx u(t_n, x_{-1}) \qquad u_{M+1}^n \approx u(t_n, x_{M+1})$$

At $i = 0$,

$$u_x(t_n, x_0) \approx \frac{u_1^n - u_{-1}^n}{2\Delta x} = 0 \; ; \quad u_{-1}^n = u_1^n \quad \text{— (2)}$$

$$u_0^{n+1} = \gamma u_{-1}^n + (1-2\gamma) u_0^n + \gamma u_1^n \quad \text{— (3)} \; ; \text{ Sub. (2) in (3)}$$

$$\boxed{u_0^{n+1} = (1-2\gamma) u_0^n + 2\gamma u_1^n}$$

Similarly for $i = M$

$$u_x(t_n, x_M) \approx \frac{u_{M+1}^n - u_{M-1}^n}{2\Delta x} = \alpha \; ;$$

$$u_{M+1}^n = u_{M-1}^n + 2\Delta x \alpha \quad \text{— (4)}$$

$$u_M^{n+1} = \gamma u_{M-1}^n + (1-2\gamma) u_M^n + \gamma u_{M+1}^n \quad \text{— (5)} \; ; \text{ Sub (4) in (5)}$$

$$\boxed{u_M^{n+1} = 2\gamma u_{M-1}^n + (1-2\gamma) u_M^n + 2\Delta x \alpha \gamma}$$

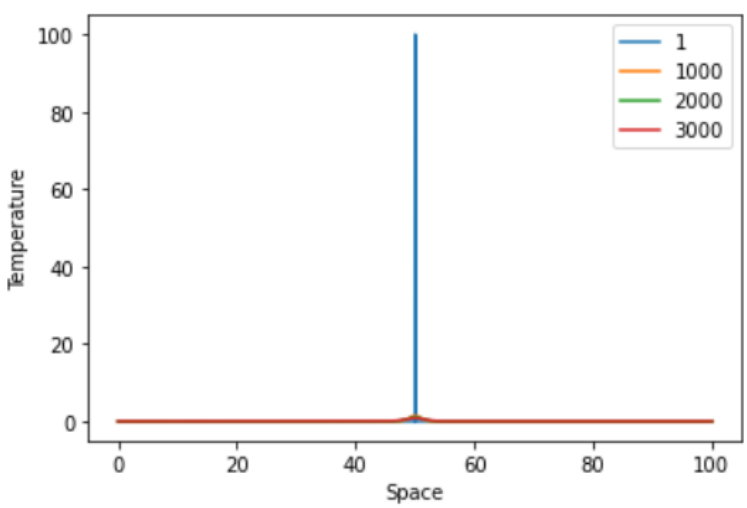
6.1 1D HEAT EQUATION – FDM AND NEUMANN BC- ZERO FLUX AT BOTH BOUNDARIES - 1

Constant initial condition and zero flux at both boundaries.

```
t_vec=np.linspace(t0,tL,N)   #creating grid
x_vec=np.linspace(x0,xL,M)   #creating grid
T=np.zeros((M,N))             #inital matrix values

bc1=float(input("Enter bounday value at x0 for any t:"))
bc2=float(input("Enter bounday value at xL for any t:"))

T[1500] = 100 #initial condition

for n in range(0,N-1):

    T[0,n+1] = (alpha*( T[1,n] - (2*dx*bc1) + T[1,n] )) + ((1-2*alpha)*T[0,n])  #i=0
    T[M-1,n+1] = (alpha*( T[M-2,n] + (2*dx*bc2) + T[M-2,n] )) + ((1-2*alpha)*T[M-1,n]) #i= M-1

    for i in range(1,M-1):
        T[i,n+1] = (alpha*( T[i-1,n] + T[i+1,n] )) + ((1-2*alpha)*T[i,n])
```

Inputs and Output (2D representation):

```
Enter No. of grid points for space interval:3000
Enter No. of grid points for time interval:3000
Enter starting space grid point:0
Enter final space grid point:100
Enter starting time grid point:0
Enter final time grid point:10
Enter Thermal Diffusivity value:0.005
Enter bounday value at x0 for any t:0
Enter bounday value at xL for any t:0
```



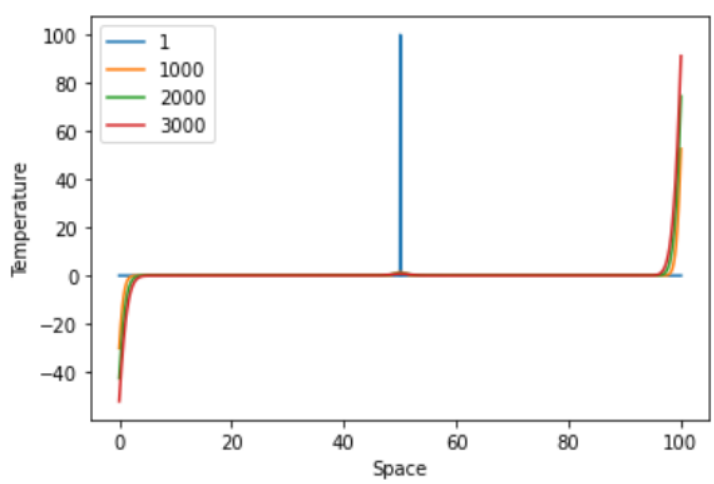
6.2 1D HEAT EQUATION – FDM AND NEUMANN BC-NON-ZERO FLUX AT BOTH BOUNDARIES - 1

Constant initial condition and non-zero flux at both boundaries.

```
t_vec=np.linspace(t0,tL,N)   #creating grid
x_vec=np.linspace(x0,xL,M)   #creating grid
T=np.zeros((M,N))             #inital matrix values

bc1=float(input("Enter bounday value at x0 for any t:"))
bc2=float(input("Enter bounday value at xL for any t:"))

T[1500] = 100 #initial condition

for n in range(0,N-1):

    T[0,n+1] = (alpha*( T[1,n] - (2*dx*bc1) + T[1,n] )) + ((1-2*alpha)*T[0,n])  #i=0
    T[M-1,n+1] = (alpha*( T[M-2,n] + (2*dx*bc2) + T[M-2,n] )) + ((1-2*alpha)*T[M-1,n]) #i= M-1

    for i in range(1,M-1):
        T[i,n+1] = (alpha*( T[i-1,n] + T[i+1,n] )) + ((1-2*alpha)*T[i,n])
```

Inputs and Output (2D representation):

```
Enter No. of grid points for space interval:3000
Enter No. of grid points for time interval:3000
Enter starting space grid point:0
Enter final space grid point:100
Enter starting time grid point:0
Enter final time grid point:10
Enter Thermal Diffusivity value:0.005
Enter bounday value at x0 for any t:40
Enter bounday value at xL for any t:70
```



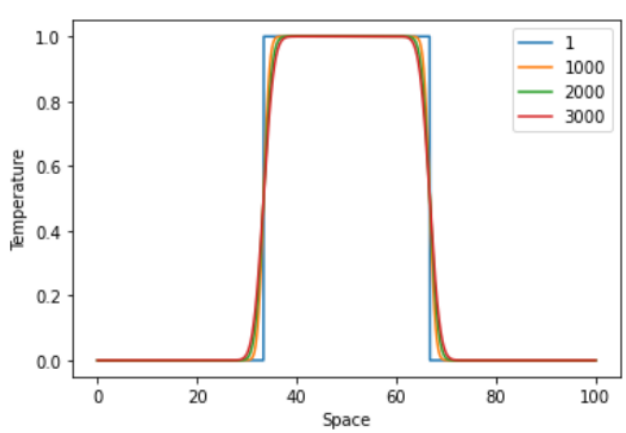
6.3 1D HEAT EQUATION – FDM AND NEUMANN BC-ZERO FLUX AT BOTH BOUNDARIES - 2

Step function as initial condition and zero flux at both boundaries.

```
t_vec=np.linspace(t0,tL,N)   #creating grid
x_vec=np.linspace(x0,xL,M)   #creating grid
T=np.zeros((M,N))             #inital matrix values

bc1=float(input("Enter bounday value at x0 for any t:"))
bc2=float(input("Enter bounday value at xL for any t:"))

for k in range(0,M):            #initial condition
    if k>=1000 and k<=2000:
        T[k,0] = 1
    else:
        T[k,0] = 0

for n in range(0,N-1):

    T[0,n+1] = (alpha*( T[1,n] - (2*dx*bc1) + T[1,n] )) + ((1-2*alpha)*T[0,n])  #i=0
    T[M-1,n+1] = (alpha*( T[M-2,n] + (2*dx*bc2) + T[M-2,n] )) + ((1-2*alpha)*T[M-1,n]) #i= M-1

    for i in range(1,M-1):
        T[i,n+1] = (alpha*( T[i-1,n] + T[i+1,n] )) + ((1-2*alpha)*T[i,n])
```

Inputs and Output (2D representation):

```
Enter No. of grid points for space interval:3000
Enter No. of grid points for time interval:3000
Enter starting space grid point:0
Enter final space grid point:100
Enter starting time grid point:0
Enter final time grid point:10
Enter Thermal Diffusivity value:0.005
Enter bounday value at x0 for any t:0
Enter bounday value at xL for any t:0
```



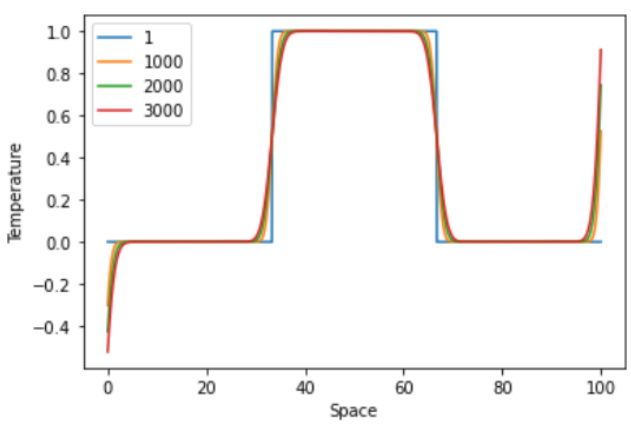
6.4 1D HEAT EQUATION – FDM AND NEUMANN BC-NON-ZERO FLUX AT BOTH BOUNDARIES - 2

Step function as initial condition and non-zero flux at both boundaries.

```
t_vec=np.linspace(t0,tL,N)   #creating grid
x_vec=np.linspace(x0,xL,M)   #creating grid
T=np.zeros((M,N))             #inital matrix values

bc1=float(input("Enter bounday value at x0 for any t:"))
bc2=float(input("Enter bounday value at xL for any t:"))

for k in range(0,M):
    if k>=1000 and k<=2000:
        T[k,0] = 1
    else:
        T[k,0] = 0

for n in range(0,N-1):

    T[0,n+1] = (alpha*( T[1,n] - (2*dx*bc1) + T[1,n] )) + ((1-2*alpha)*T[0,n])  #i=0
    T[M-1,n+1] = (alpha*( T[M-2,n] + (2*dx*bc2) + T[M-2,n] )) + ((1-2*alpha)*T[M-1,n]) #i= M-1

    for i in range(1,M-1):
        T[i,n+1] = (alpha*( T[i-1,n] + T[i+1,n] )) + ((1-2*alpha)*T[i,n])
```

Inputs and Output (2D representation):

```
Enter No. of grid points for space interval:3000
Enter No. of grid points for time interval:3000
Enter starting space grid point:0
Enter final space grid point:100
Enter starting time grid point:0
Enter final time grid point:10
Enter Thermal Diffusivity value:0.005
Enter bounday value at x0 for any t:0.4
Enter bounday value at xL for any t:0.7
```



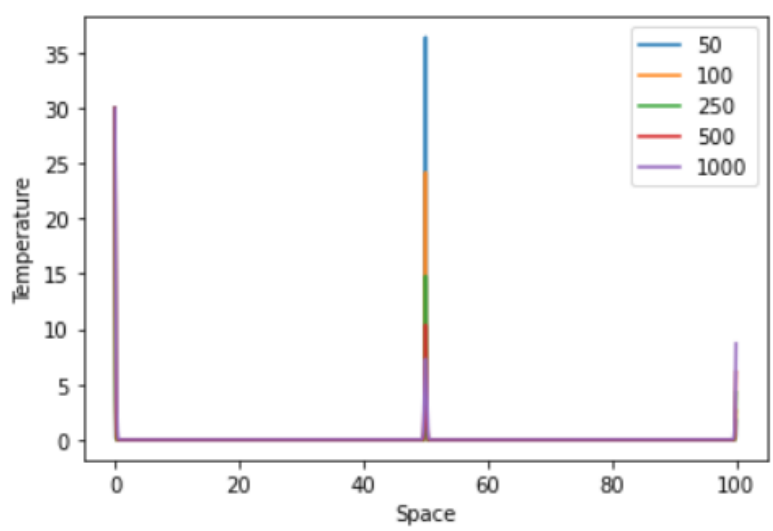
7. 1D HEAT EQUATION – FDM AND MIXED BC (DIRICHLET AND NEUMANN)

Constant initial condition and we apply Dirichlet BC at one end and Neumann BC at another end.

```
t_vec=np.linspace(t0,tL,N)   #creating grid
x_vec=np.linspace(x0,xL,M)   #creating grid
T=np.zeros((M,N))             #inital matrix values

bc1=float(input("Enter bounday value at x0 for any t:"))
bc2=float(input("Enter bounday value at xL for any t:"))
T[0,:]=bc1

T[1500] = 100    #inital condition

for n in range(0,N-1):
    T[M-1,n+1] = (alpha*( T[M-2,n] + (2*dx*bc2) + T[M-2,n] )) + ((1-2*alpha)*T[M-1,n]) #i=M-1
    for i in range(1,M-1):
        T[i,n+1] = (alpha*( T[i-1,n] + T[i+1,n] )) + ((1-2*alpha)*T[i,n])
```

Input and Output (2D representation):

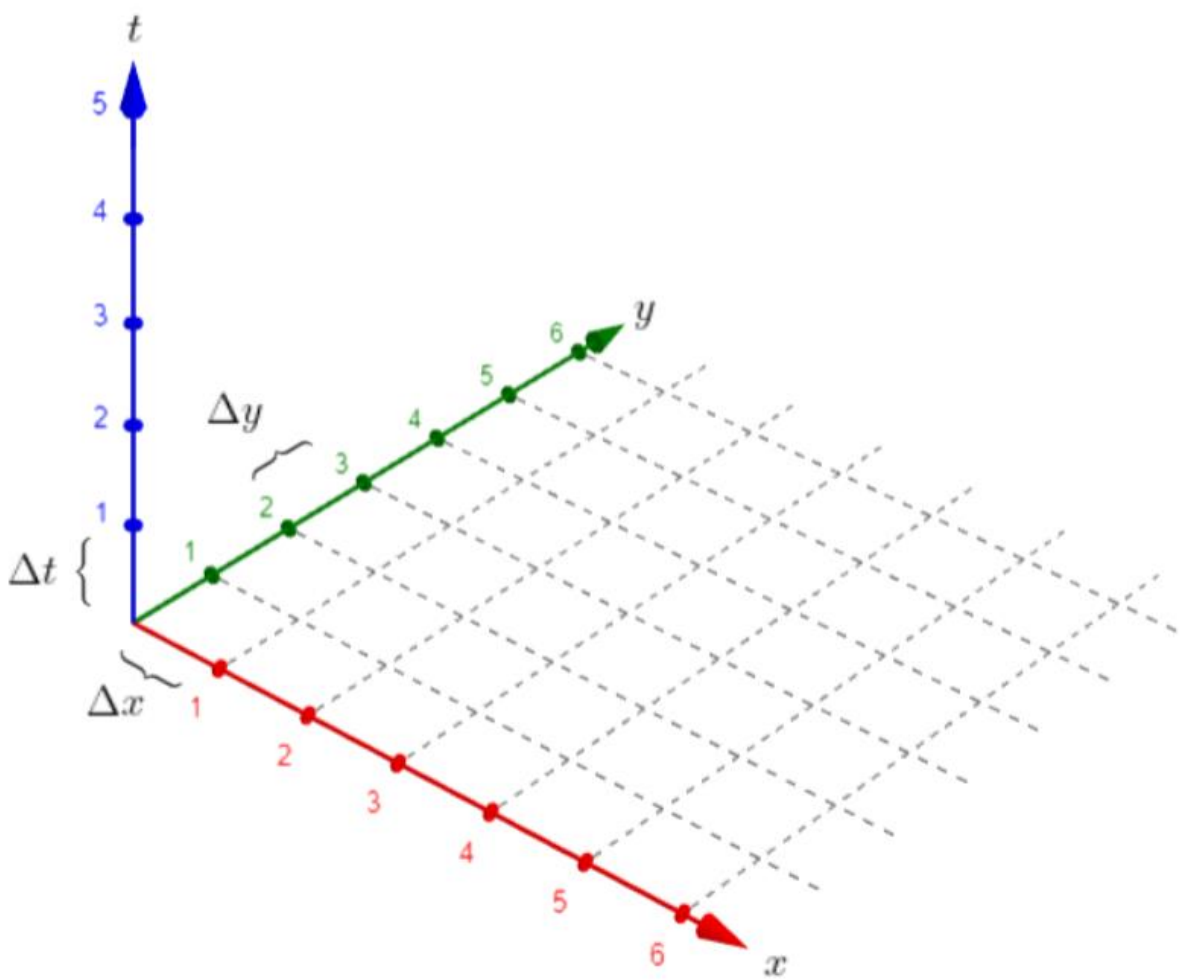
```
Enter No. of grid points for space interval:3000
Enter No. of grid points for time interval:3000
Enter starting space grid point:0
Enter final space grid point:100
Enter starting time grid point:0
Enter final time grid point:10
Enter Thermal Diffusivity value:0.005
Enter bounday value at x0 for any t:30
Enter bounday value at xL for any t:60
```



8. THE 2D HEAT EQUATION

$$\frac{\partial u}{\partial t} = D\left(\frac{\partial^2 u}{\partial x^2} + \frac{\partial^2 u}{\partial y^2}\right),$$

- We discretize the spatial domain '*x*', '*y*' and the time interval'*t*'.



$$x_i = i\Delta x$$
$$y_j = j\Delta y$$
$$t_k = k\Delta t$$

$t_n = t_0 + k\Delta t$, k = 0,1,2,…

$x_n = x_0 + i\Delta x$, i = 0,1,…M

$y_n = y_0 + j\Delta y$, j = 0,1,…N

- i, j, and k are the steps for each difference for x, y, and t respectively.

$$u(x, y, t) = u_{i,j}^{k}$$

- The 2D heat equation can be written using the FDM as :

$$\frac{u_{i,j}^{k+1} - u_{i,j}^{k}}{\Delta t} - \alpha \left(\frac{u_{i+1,j}^{k} - 2u_{i,j}^{k} + u_{i-1,j}^{k}}{\Delta x^2} + \frac{u_{i,j+1}^{k} - 2u_{i,j}^{k} + u_{i,j-1}^{k}}{\Delta y^2} \right) = 0$$

Arranging the above equation above by taking $\Delta x = \Delta y$, we get

$$u_{i,j}^{k+1} = \gamma(u_{i+1,j}^{k} + u_{i-1,j}^{k} + u_{i,j+1}^{k} + u_{i,j-1}^{k} - 4u_{i,j}^{k}) + u_{i,j}^{k}$$

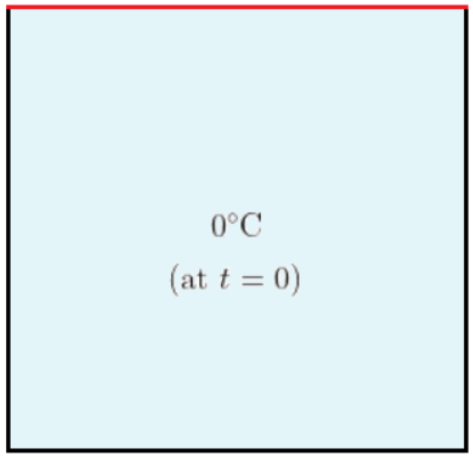
where,

$$\gamma = \alpha \frac{\Delta t}{\Delta x^2}$$

Explicit method has been used to get the solution for the heat equation, so it will be numerically stable whenever

$$\Delta t \leq \frac{\Delta x^2}{4\alpha}$$

8.1 2D HEAT EQUATION – FDM AND DIRICHLET BC



u_top = 100.0
u_left = 20.0
u_bottom = 10.0
u_right = 0.0

- First we set up the initial and boundary conditions.

```
# Initialize solution: the grid of u(k, i, j)
u = np.empty((max_iter_time, plate_length, plate_length))

# Initial condition everywhere inside the grid
u_initial = 10

# Boundary conditions
u_top = 100.0
u_left = 20.0
u_bottom = 10.0
u_right = 0.0

# Setting the initial condition
u.fill(u_initial)

# Setting the boundary conditions
u[:, (plate_length-1):, :] = u_top
u[:, :, :1] = u_left
u[:, :1, 1:] = u_bottom
u[:, :, (plate_length-1):] = u_right
```

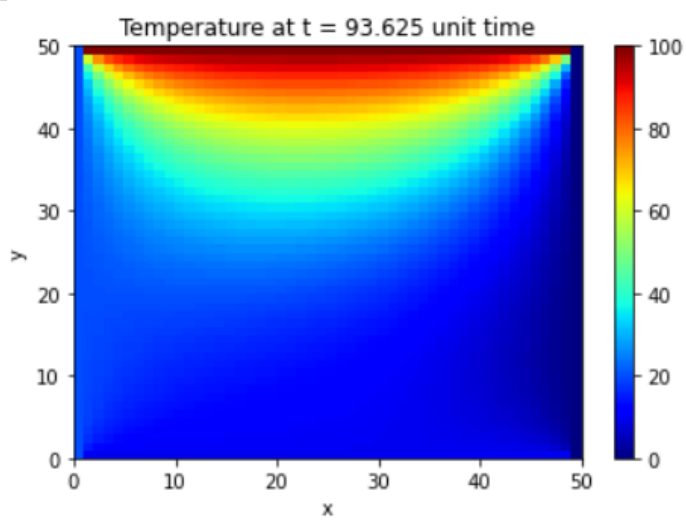
- Next we write the calculation function based on finite-difference method.

```
def calculate(u):
    for k in range(0, max_iter_time-1, 1):
        for i in range(1, plate_length-1, delta_x):
            for j in range(1, plate_length-1, delta_x):
                u[k + 1, i, j] = gamma * (u[k][i+1][j] + u[k][i-1][j] + u[k][i][j+1] + u[k][i][j-1] - 4*u[k][i][j]) + u[k][i][j]

    return u
```

- We then use the plot function to visualize the solution (for each K) as a heat map. We also animate the result to see the temperature points inside the plate change over time.

```
def plotheatmap(u_k, k):

    plt.clf()

    plt.title(f"Temperature at t = {k*delta_t:.3f} unit time")
    plt.xlabel("x")
    plt.ylabel("y")

    # This is to plot u_k (u at time-step k)
    plt.pcolormesh(u_k, cmap=plt.cm.jet, vmin=0, vmax=100)
    plt.colorbar()

    return plt


u = calculate(u)

def animate(k):
    plotheatmap(u[k], k)

anim = animation.FuncAnimation(plt.figure(), animate, interval=1, frames=max_iter_time, repeat=False)
anim.save("heat_equation_solution_1.gif")
```

Output:



8.2 2D HEAT EQUATION – FDM AND DIRICHLET BC (CODE 2)

- We try to set up/initialize the temperature matrix, it has grid points of x, y, and t as its attributes.

```
import numpy as np
import matplotlib.pyplot as plt

w = int(input("Enter length of the material:")) # length along x axis
h = int(input("Enter width of the material:"))  # length along y axis
t = int(input("Enter total time interval:"))    # total run time in ms

Nx = int(input("Enter number of grid points for length:"))
Ny = int(input("Enter number of grid points for width:"))
Nt = int(input("Enter number of grid points for time:"))

x_vec=np.linspace(0,w,Nx)
y_vec=np.linspace(0,h,Ny)

dx = x_vec[1] - x_vec[0]
x2 = dx**2

dy = y_vec[1] - y_vec[0]
y2 = dy**2

D1 = int(input("Enter Thermal Diffusitivity value:"))
D = D1/100000 # Thermal Diffusitivity values are usually low

dt = (x2*y2)/(2*D*(x2+y2)) # maximum time step for stability condition

kx = (D*dt/x2)
ky = (D*dt/y2)

T = np.zeros((Ny,Nx,Nt))  # 3d array
```

- Now we set up the initial and boundary conditions.

```
ic = int(input("Enter initial condition:"))
T[:,:,0] = ic

top = int(input("Enter temp value at top:"))
bottom = int(input("Enter temp value at bottom:"))
left = int(input("Enter temp value at left:"))
right = int(input("Enter temp value at right:"))

T[Nx-1,:,:] = top
T[0,:,:] = bottom

T[:,0,:] = left
T[:,Ny-1,:] = right
```

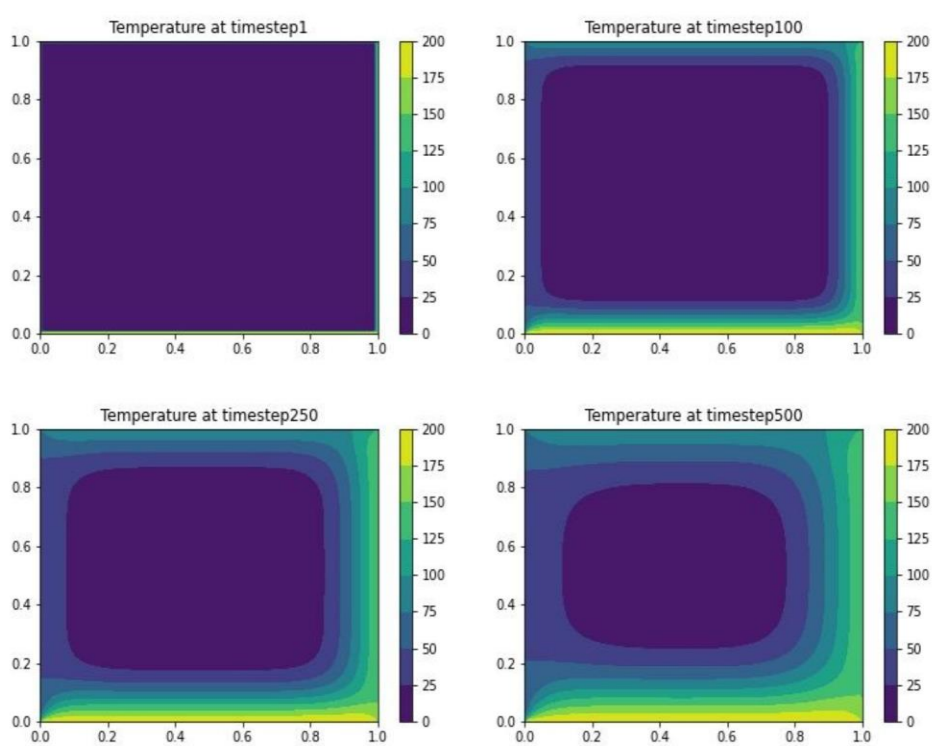
- We then write the calculation function based on FDM.

```
for k in range(1,Nt):
    for i in range(1,Nx-1):
        for j in range(1,Ny-1):
            H = kx*(T[j,i+1,k-1] + T[j,i-1,k-1] - (2*T[j,i,k-1]))
            V = ky*(T[j+1,i,k-1] + T[j-1,i,k-1] - (2*T[j,i,k-1]))
            T[j,i,k] = T[j,i,k-1] + H + V
```

- Next we try to visualize the solution in the form of a plot and animate it to understand the evolution at each time step.

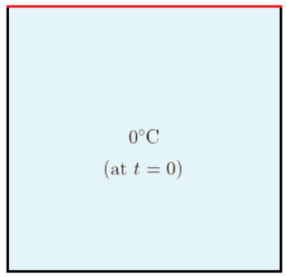
```
for k in range(0,Nt):
    plt.contourf(x_vec, y_vec, T[:,:,k])
    plt.colorbar()
    plt.title('Temperature at timestep' + str(k+1))
    plt.savefig(r'C:\Users\admin\OneDrive\Desktop\2D Heat Equation - DN BC'+ '\\' + str(k+1) + '.jpg')
    # save Temp profile as jpg at each timestep
    plt.clf()

import PIL  # python imaging library

image_frames = []

for k in range(0,Nt):
    new_frame = PIL.Image.open(r'C:\Users\admin\OneDrive\Desktop\2D Heat Equation - DN BC' + '\\' + str(k+1) +'.jpg')
    image_frames.append(new_frame)  # merging the images to form a GIF

image_frames[0].save(r'C:\Users\admin\OneDrive\Desktop\2D Heat Equation - DN BC' + 'ND_evolution.gif', format='GIF',
                     append_images = image_frames[1:],
                     save_all = True, duration=t, loop=0)   # saving the GIF
```

Output:



Evolution of temperature at different time steps

8.3 2D HEAT EQUATION – FDM AND MIXED BC (DIRICHLET AND NEUMANN)

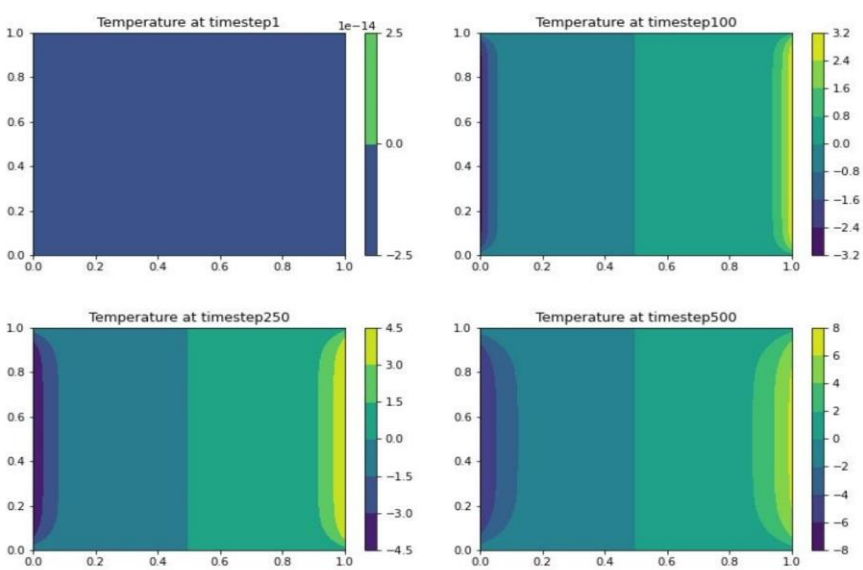


u_top (Dirichlet) = 0.0
u_left (Neumann) = 50.0
u_bottom (Dirichlet) = 0.0
u_right (Neumann) = 50.0

- We follow the same technique used for Dirichlet BC (Code 2) but there is a change in the calculation function loop alone for the Dirichlet – Neumann mixed BC.

```
top = int(input("Enter temp value at top:"))
bottom = int(input("Enter temp value at bottom:"))
left = int(input("Enter dT/dx value at left:"))
right = int(input("Enter  dT/dx value at right:"))

T[Nx-1,:,:] = top
T[0,:,:] = bottom

for k in range(1,Nt):
    for i in range(1,Nx-1):
        for j in range(1,Ny-1):
            T[j,0,k] = (kx*(2*T[j,1,k-1] - (2*dx*left) - (2*T[j,0,k-1])) + T[j,0,k-1] +
                        ky*(T[j+1,0,k-1] + T[j-1,0,k-1] - (2*T[j,0,k-1])))
            H = kx*(T[j,i+1,k-1] + T[j,i-1,k-1] - (2*T[j,i,k-1]))
            V = ky*(T[j+1,i,k-1] + T[j-1,i,k-1] - (2*T[j,i,k-1]))
            T[j,i,k] = T[j,i,k-1] + H + V
            T[j,Ny-1,k] = (kx*(2*T[j,Ny-2,k-1] + (2*dx*right) - (2*T[j,Ny-1,k-1])) + T[j,Ny-1,k-1] +
                           ky*(T[j+1,Ny-1,k-1] + T[j-1,Ny-1,k-1] - (2*T[j,Ny-1,k-1])))
```

Output:



Evolution of temperature at different time steps.

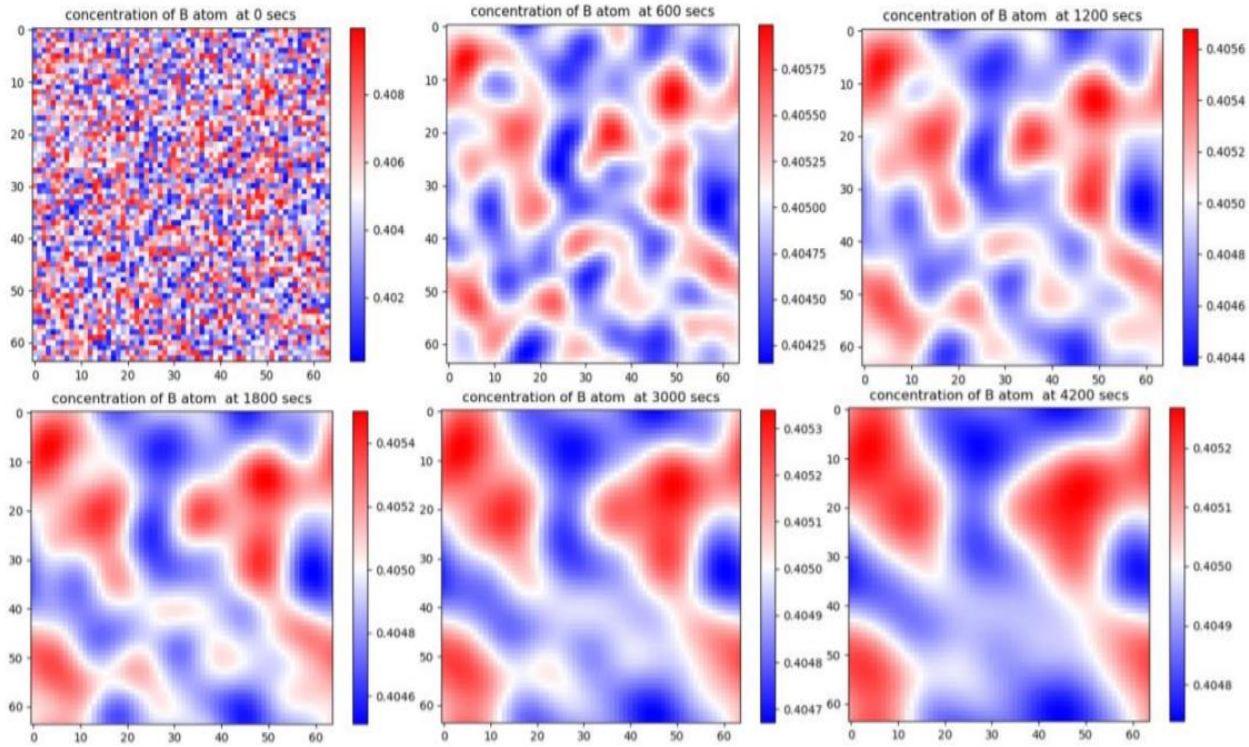
9. INTRODUCTION TO PHASE FIELD MODELLING

- Phase field modelling technique is critical to contextualizing material microstructures and to represent the composition of microstructural evolution.

- To numerically model the phase field, the explicit finite difference and spectral methods were found to be several orders of magnitude more stable.

- The finite-difference algorithm offers a more direct approach to the numerical solution of partial differential equations than any other method.

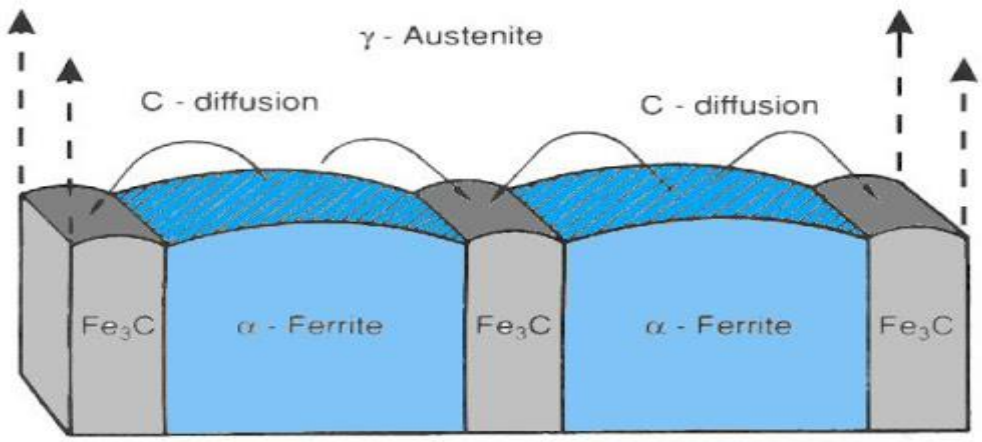
- An example of the simulation and results:



10. DISSOLUTION OF PEARLITE

- Pearlite is a eutectoid mixture of 2 phases : Ferrite and Cementite.

- It possesses a lamellar structure and consists of alternating layers of ferrite (87.5 wt%) and cementite (12.5 wt%).



Structure of Pearlite lamellae

- These layers are in direct contact with each other. The intermetallic compound cementite has 6.67 weight percent carbon, whereas ferrite is almost pure iron, free of carbon.

- Though a carbon gradient exists, the structure is thermodynamically stable at room temperature. As the temperature increases to higher values, but

below eutectoid temperature, carbon atoms have a tendency to diffuse into ferrite.

- Maximum content of carbon in ferrite is about 0.025% at eutectoid temperature.

- On heating to eutectoid temperature, BCC iron lattice changes to FCC iron lattice.

- The maximum solubility of carbon in FCC iron is about 2 percent. Therefore, at this temperature, regions around the cementite layer will be enriched with carbon because of diffusion.

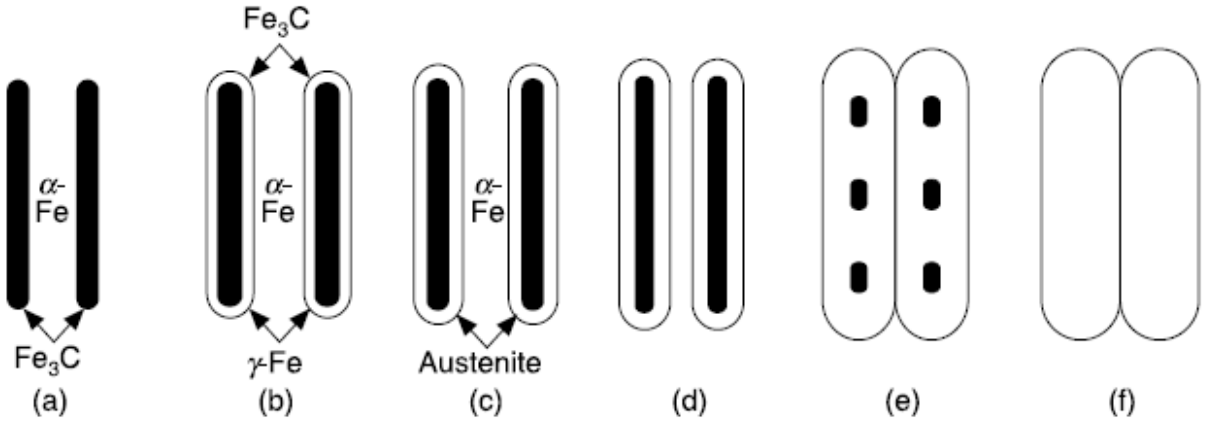
- The maximum diffusion of carbon atoms will take place from the cementite at the ferrite-cementite interface.

- As sufficient number of interfaces are available, austenite nuclei will be formed at the interfaces.

- By gradual dissolution of carbon of cementite into gamma iron, primary austenitic grains are formed.

- These primary austenitic grains dissolve the surrounding ferrite and austenitic grains grow at the expense of ferrite.

Steps associated with transformation of Pearlite to austenite



FICK'S LAW OF DIFFUSION

Change in concentration of diffusant with time

$$\frac{dC}{dt} = D\frac{d^2C}{dx^2}$$

where,

C is the concent
D is the diffusion coefficient

We use the Fick's law of diffusion as the PDE for numerical modelling of pearlite dissolution. We employ the same technique that we used for evolution of temperature profile in a 2D plate with appropriate BCs.

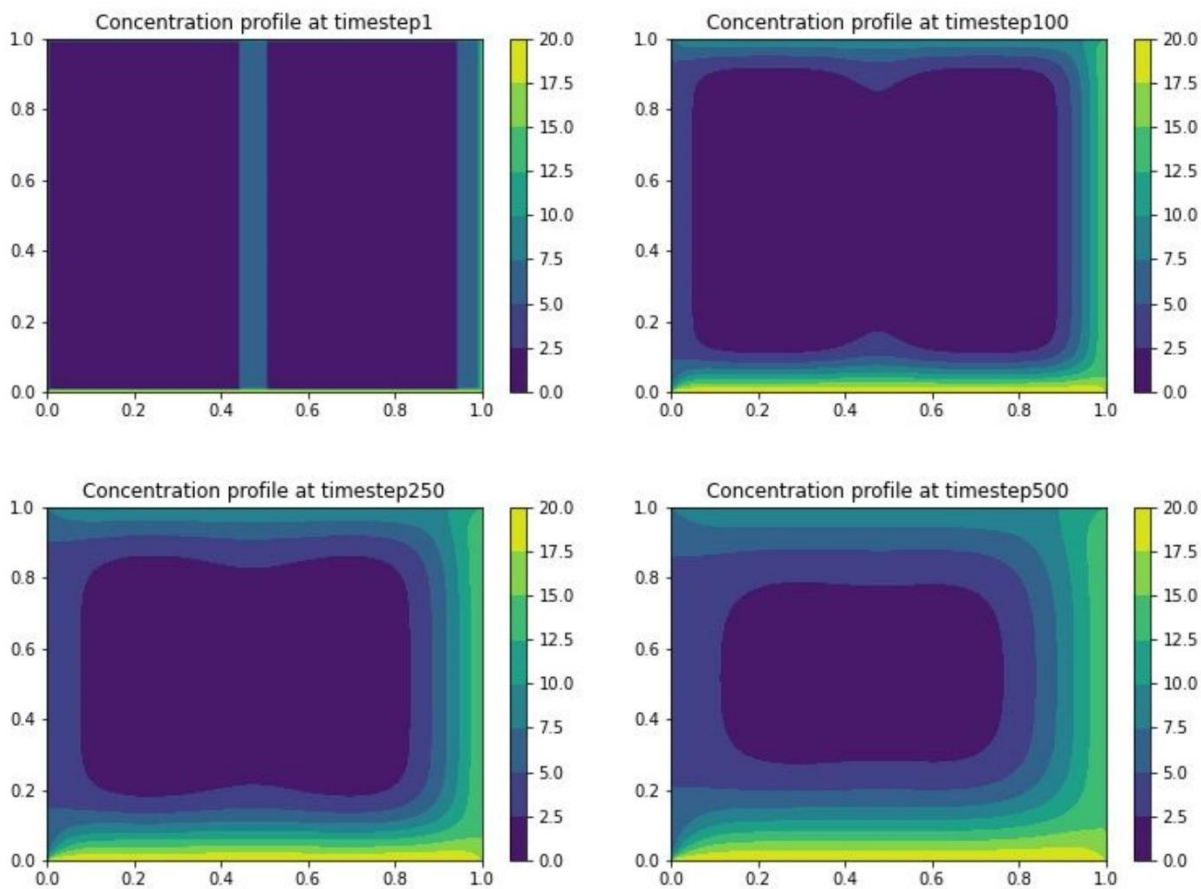
10.1 NUMERICAL MODELLING OF PEARLITE DISSOLUTION - DIRICHLET BOUNDARY CONDITION

- We implement two pearlite lamellae in our codes.
- The division by volume of the number of grids for Ferrite and Cementite is done appropriately.
- In pearlite, the ferrite constitutes to 88 wt% and cementite to 12 wt%.
- So we consider the first 44% grids along the x-axis as Ferrite and the next 6% as Cementite. This constitutes to a single pearlite lamellae.
- Similarly, the next 44% of the second lamellae is Ferrite, and the remaining 6% is Cementite, thus totally making up 100% of our 2 pearlite system.
- The method of approach is similar to the modelling of 2D heat equation with Dirichlet boundary condition.
- The only difference is that there we find the evolution of temperature at different time steps. In this case we find the evolution/change in carbon concentration at different time steps.

- We set up the initial and the boundary conditions as shown below.

```
top = int(input("Enter Carbon conc. value at top:"))
bottom = int(input("Enter Carbon conc. value at bottom:"))
left = int(input("Enter Carbon conc. value at left:"))
right = int(input("Enter Carbon conc. value at right:"))

T = np.zeros((Ny,Nx,Nt))  # 3d array

for c in range(0,Nx):
    if ((c>=0 and c<(0.44*Nx)) or (c>=(0.5*Nx) and c<(0.94*Nx))):
        T[:,c,0] = 0.02 # ferrite carbon conc.
    else:
        T[:,c,0] = 6.67 # cementite carbon conc.

T[Nx-1,:,:] = top
T[0,:,:] = bottom

T[:,0,:] = left
T[:,Ny-1,:] = right
```

Output:



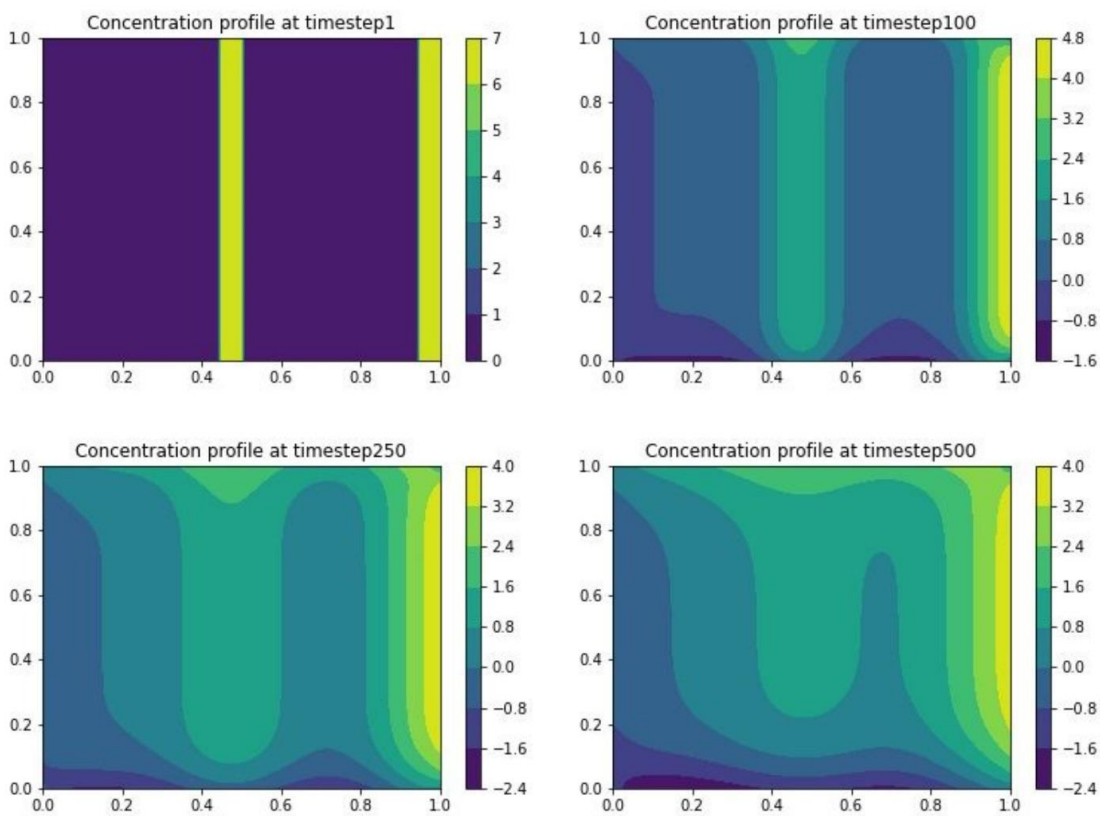
Evolution of carbon concentration at different time steps

10.2 NUMERICAL MODELLING OF PEARLITE DISSOLUTION - NEUMANN BOUNDARY CONDITION

We use the same approach as that for Dirichlet BC. There is only change in the calculation function loop where FDM for Neumann condition is employed.

```
for k in range(1,Nt):
    for i in range(1,Nx-1):
        for j in range(1,Ny-1):
            T[j,0,k] = (kx*(2*T[j,1,k-1] - (2*dx*left) - (2*T[j,0,k-1])) + T[j,0,k-1] +
                        ky*(T[j+1,0,k-1] + T[j-1,0,k-1] - (2*T[j,0,k-1])))

            T[0,i,k] = (ky*(2*T[1,i,k-1] - (2*dx*bottom) - (2*T[0,i,k-1])) + T[0,i,k-1] +
                        kx*(T[0,i+1,k-1] + T[0,i-1,k-1] - (2*T[0,i,k-1])))

            H = kx*(T[j,i+1,k-1] + T[j,i-1,k-1] - (2*T[j,i,k-1]))
            V = ky*(T[j+1,i,k-1] + T[j-1,i,k-1] - (2*T[j,i,k-1]))
            T[j,i,k] = T[j,i,k-1] + H + V

            T[j,Ny-1,k] = (kx*(2*T[j,Ny-2,k-1] + (2*dx*right) - (2*T[j,Ny-1,k-1])) + T[j,Ny-1,k-1] +
                           ky*(T[j+1,Ny-1,k-1] + T[j-1,Ny-1,k-1] - (2*T[j,Ny-1,k-1])))

            T[Nx-1,i,k] = (ky*(2*T[Nx-2,i,k-1] + (2*dx*top) - (2*T[Nx-1,i,k-1])) + T[Nx-1,i,k-1] +
                           kx*(T[Nx-1,i+1,k-1] + T[Nx-1,i-1,k-1] - (2*T[Nx-1,i,k-1])))
```

Output:



Evolution of carbon concentration at different time steps

BIBLIOGRAPHY

©2010 Professor D. M. Causon, Professor C.G. Mingham & Ventus Publishing ApS
Introductory Finite Difference Methods for PDEs
ISBN 978-87-7681-642-1

Hans Petter Langtangen, Svein Linge
Finite Difference Computing with PDEs - A Modern Software Approach,
ISSN 1611-0994
Texts in Computational Science and Engineering
ISBN 978-3-319-55455-6 ISBN 978-3-319-55456-3 (eBook)
DOI 10.1007/978-3-319-55456-3

Sandip Mazumder, The Ohio State University
Numerical Methods for Partial Differential Equations -Finite Difference and Finite Volume Methods
ISBN: 978-0-12-849894-1

T.V. Rajan, C.P. Sharma and Ashok Sharma
HEAT TREATMENT: Principles and Techniques, Second Edition
PHI Learning Private Limited
ISBN-978-81-203-4095-4